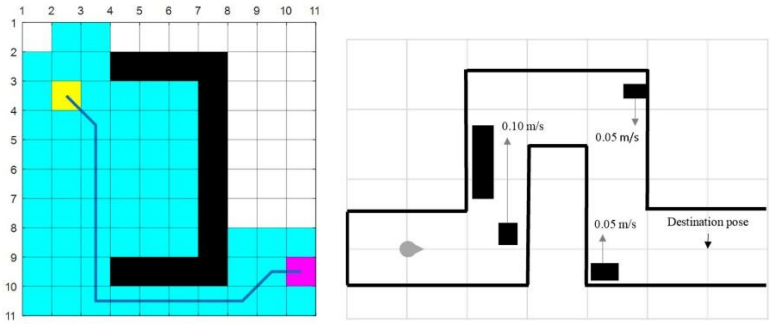
1) Path Planning Fundamentals

•Difference between a path and a trajectory. • Waypoint, obstacle, free space, start state, and goal.

. What is path planning?

Imagine an autonomous ground vehicle driving through an environment. It knows where it starts, where it needs to go, and which areas are blocked. Its job is to find a route from its starting point to its destination without hitting obstacles.

1.1 Path versus trajectory



Path = where to go

A sequence of positions connecting the start to the goal.

Trajectory = where to go and when

A path with timing and motion information, such as speed or velocity at different positions.

A path describes spatial movement; a trajectory also specifies how the movement evolves over time. In robotics, a trajectory may additionally include velocity and acceleration constraints.

1.2)

Start state	The initial position or condition of the vehicle.
Goal	The desired destination or final condition.
Waypoint	An intermediate position the vehicle may pass through.
Obstacle	Something the vehicle must avoid.
Free space	The area in which the vehicle is allowed to move.

A state can mean more than position in a real robot. It could also include orientation and velocity. In our beginner project, we simplify the state to just a grid coordinate: (row, column).

2. Environment Representation

• What is a 2D grid map? • What is an occupancy grid? • Explain free, occupied, and unknown cells

2.1 What is a 2D grid map?

Imagine dividing a floor into rows and columns, like a chessboard.

In Python, we can represent the same idea using a NumPy array:

grid = [[0, 0, 0, 1, 0], [0, 1, 0, 1, 0], [0, 0, 0, 0, 0], [1, 1, 0, 1, 0]]

Here:

- 0 means free.
- 1 means occupied by an obstacle.

We will store the start and goal separately.

A grid is convenient because the computer can represent each cell with a pair of integers and examine neighbouring cells.

2.2 What is an occupancy grid?

An occupancy grid is a grid in which each cell represents how likely it is to be occupied by an obstacle.

Free cell The area is considered clear, so the planner may travel through it.

Occupied cell An obstacle is present, so the planner must avoid it.

Unknown cell The robot does not yet know whether the area is free or occupied. A conservative planner may treat it as blocked until it is observed.

In a real robot, sensors such as LiDAR or cameras can help update the occupancy map.

A simple grid map says whether a cell is free or blocked, while a probabilistic occupancy grid can represent uncertainty about occupancy.

3. Search Algorithms

• BFS (Breadth-First Search) — working principle and intuition. • Dijkstra's Algorithm — working principle and how it differs from BFS when movement costs are not equal. • A* Algorithm — working principle and the role of the heuristic.

3.1 BFS — Breadth-First Search

BFS behaves similarly: it explores all locations one step away, then all locations two steps away, then three, and so on.



How it works:

1. Put the start cell into a queue.
2. Remove the oldest cell from the queue.
3. Examine its neighbours.
4. Add every valid, previously unvisited neighbour to the queue.
5. Record which cell each newly discovered cell came from.
6. Continue until the goal is reached or the queue becomes empty.
7. Follow the recorded parents backwards to reconstruct the path.

A queue works on FIFO: first in, first out. The first cell added is the first one processed.

Why does BFS find the shortest path? When every move costs the same, it explores cells in increasing order of the number of moves required to reach them.

Its limitation is that it does not naturally prefer a cheaper route when different moves have different costs.

3.2 Dijkstra's algorithm

Imagine that not every part of the ground is equally easy to cross.

- Normal ground costs 1.
- Mud costs 5.
- A rough patch costs 10.

The shortest route in terms of the number of cells may not be the cheapest route in terms of movement cost.

Dijkstra's algorithm accounts for this by maintaining the total cost from the start.

It repeatedly processes the cell with the lowest known cost, using a priority queue rather than BFS's ordinary queue.

For each neighbour, it calculates:

$$\text{new cost} = \text{current cost} + \text{move cost}$$

If the new cost is lower than the previously recorded cost, it updates that neighbour's cost and parent.

BFS versus Dijkstra:

Feature	BFS	Dijkstra
Queue	Ordinary FIFO queue	priority queue
Movement cost	Assumes equal cost	Supports different non-negative costs
Main priority	Fewest steps	Lowest accumulated cost
Best use	Simple, unweighted grids	Weighted maps

If all moves cost 1, BFS and Dijkstra can find the same shortest paths, though the exact route may differ when multiple equally short routes exist.

3.3 A* — A-star search

A* builds on Dijkstra but adds an estimate of how far each cell is from the goal.

This estimate is called the heuristic.

Think of it this way:

- Dijkstra asks: “How much have I spent getting here?”
- A* asks: “How much have I spent getting here, and how far do I still appear to be from the destination?”

A* calculates a priority:

$$\boxed{f(n) = g(n) + h(n)}$$

Where:

- g(n)= actual cost from the start to cell \(n\).
- h(n) = estimated cost from cell \(n\) to the goal.
- f(n) = combined priority used to decide which cell to explore next.

For four-directional movement with each move costing 1, we use Manhattan distance:

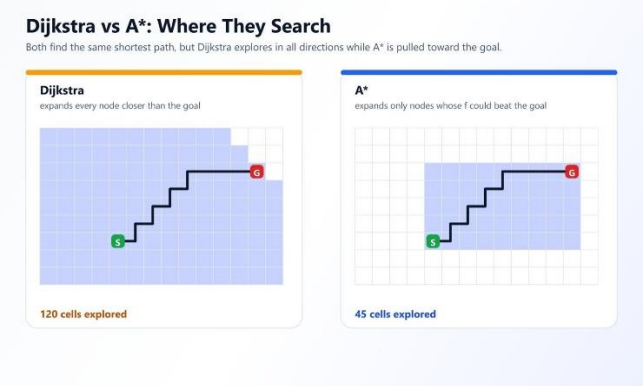
$$h(n) = |r_n - r_g| + |c_n - c_g|$$

For example, if the current cell is (2, 3) and the goal is (7, 8), the heuristic is:

h=|2-7|+|3-8|=10

This means at least 10 moves are needed if movement is restricted to four directions, regardless of obstacles.

Why the heuristic matters



Dijkstra Expands based on distance already travelled. It can explore many cells in directions that do not lead toward the goal.

A*Uses the estimated remaining distance to guide exploration toward the destination, often examining fewer cells.

A* can find an optimal path when the heuristic is admissible (it never overestimates the true remaining cost) and the search is implemented correctly. Our Manhattan heuristic meets this condition for a four-directional grid with unit movement costs.

4. Global and Local Planning

• Difference between a global path and a local motion decision. • Why an autonomous vehicle may need to re-plan when a new obstacle appears.

Global path planning

The planner uses the available map to find a route from the start to the goal.It considers the larger environment and searches for a route through it.

Local motion planning

The vehicle considers what is happening around it right now and decides how to move safely in the immediate future.For example, it may need to slow down, stop, steer around a pedestrian, or choose a temporary detour.

Global planning	Local planning
Looks at the overall route	Focuses on nearby movement and immediate conditions
Often uses a map	Uses nearby observations and current vehicle state
Produces a route toward the goal	Chooses safe short-term motions

May become outdated if the map changes Can respond to newly observed obstacles

Why re-planning matters?

Suppose A* finds a route through a cell that was initially free. A new obstacle appears in that cell.The old route may now be unsafe. The planner needs an updated map and must search again for a valid route.